LAPORAN MILESTONE 1 - LEXICAL ANALYSIS

TUGAS BESAR ARION COMPILER

IF2224 TEORI BAHASA FORMAL DAN AUTOMATA

Disusun oleh:

K1 - Kelompok 13: ToBeFairO

13524003 - Faiq Azzam Nafidz
13524013 - Anindya Naufal Pinasthika
13524083 - Mahmudia Kimdaro Amin
13524095 - Jingglang Galih Rinenggan

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2026

Daftar Isi

1. Teori Singkat.....3
1.1. Deterministic Finite Automata (DFA).....3
1.2. Lexical Analysis (Lexer).....3
1.3. Token.....4
2. Perancangan & Implementasi.....5
2.1. Desain Diagram Transisi Automata.....5
3. Pengujian.....12
3.1. Test Case 1.....12
3.2. Test Case 2.....13
3.3. Test Case 3.....14
3.4. Test Case 4.....16
3.5. Test Case 5.....17
4. Kesimpulan dan Saran.....20
4.1. Kesimpulan.....20
4.2. Saran.....20
5. Lampiran.....21
5.1. Github.....21
5.2. Diagram DFA.....21
5.3. Tabel Pembagian Tugas.....21
6. Referensi.....22

1. Teori Singkat

1.1. Deterministic Finite Automata (DFA)

Merupakan mesin automata dari suatu ekspresi bahasa yang diekspresikan dalam bentuk *state* dan *transition* di mana DFA memiliki karakteristik bahwa mesin automata tersebut memiliki satu arah *state* tujuan tanpa ada ambiguitas dalam *transition*-nya.

Secara Formal, didefinisikan sebagai 5-tuple:

$M = (Q, \Sigma, \delta, q0, F)$

dalam hal ini

- Q = himpunan state yang terbatas
- Σ = himpunan simbol input (alfabet)
- $\delta : Q \times \Sigma \rightarrow Q$ = fungsi transisi
- $q0 \in Q$ = state awal (start state)
- $F \subseteq Q$ = himpunan state akhir (final states atau accepting states)

1.2. Lexical Analysis (Lexer)

Merupakan sebuah rangkaian proses mengubah suatu input file berupa string teks menjadi sebuah rangkaian token-token. Teknik untuk melakukan sebuah lexical analysis disebut lexical analyzer (atau biasa disebut lexer). Lexer akan memecah sebuah string menjadi sebuah token-token yang nantinya dapat diproses lebih lanjut oleh compiler. Lexer menggunakan konsep DFA untuk membaca dan memproses input string yang diberikan sehingga akan menghasilkan token yang diharapkan. Hasil unit atom yang muncul dari rangkaian proses lexer disebut lexeme.

Contoh:

Masukkan:

```
5
```

Akan diubah oleh lexer menjadi:

```
intcon(5)
```

Di mana intcon melambangkan sebuah tipe token, sedangkan “5” merupakan sebuah lexeme.

1.3. Token

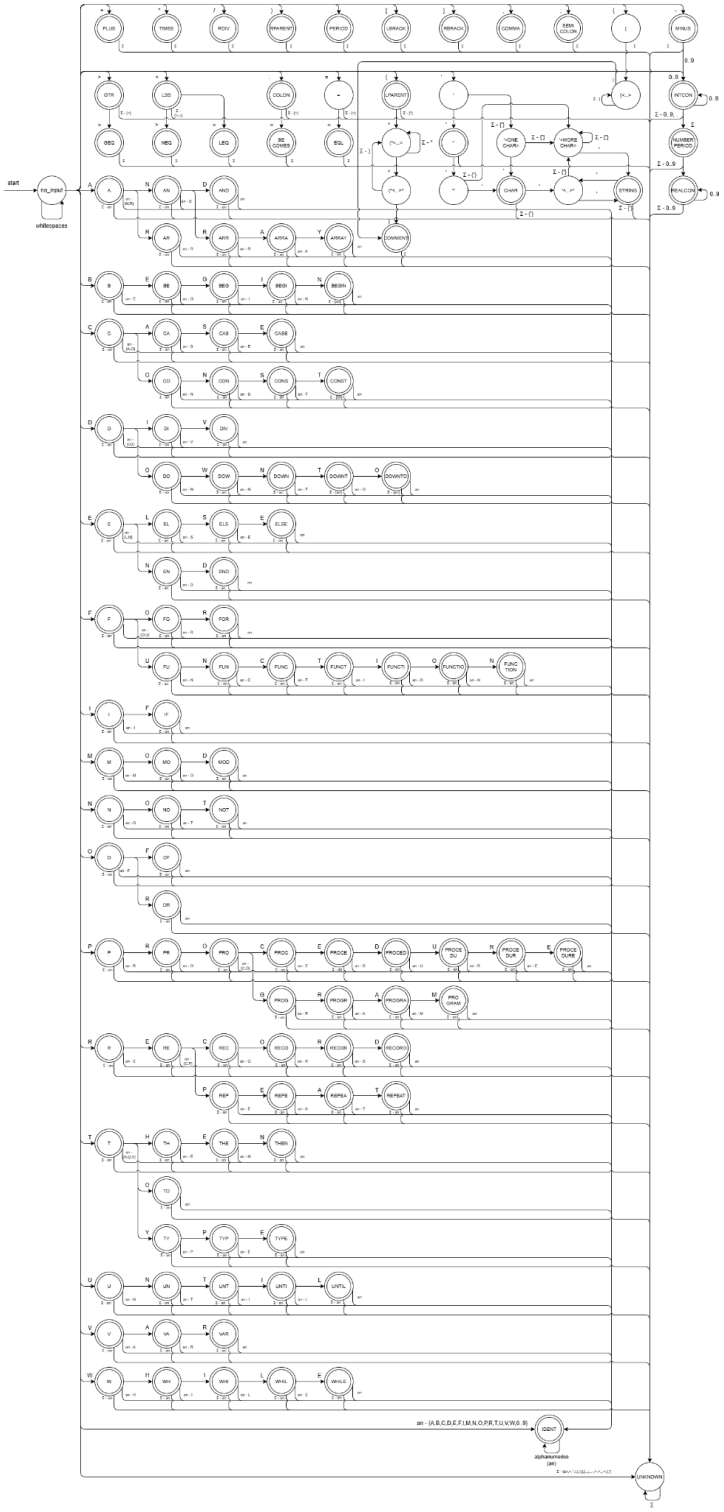
Token adalah unit terkecil yang memiliki makna dalam sebuah program. Token dibagi menjadi 2 bagian utama:

1. Token Type (Tipe Token)
   Kategori atau kelas dari token tersebut.
   Contoh: KEYWORD, IDENTIFIER, NUMBER, OPERATOR, SYMBOLS.
2. Token Value (Nilai Token)
   Sekuens karakter aktual yang membentuk token tersebut (disebut juga lexeme).
   Contoh: x, y, -, 67

2. Perancangan & Implementasi

2.1. Desain Diagram Transisi Automata

Berikut adalah diagram transisi automata (DFA) untuk lexer yang dapat menentukan pola karakter dan menghasilkan token terhadap masukan string yang akhirannya tidak memiliki whitespace (bisa dilihat juga melalui DFA 13.png).



Gambar 1. Diagram Transisi Automata terhadap *Lexer*

Di DFA tersebut terdapat *state* biasa, *dead state*, dan *final state* yang masing-masing memberikan keluaran yang berbeda dalam program jika DFA berhenti pada *state* terkait. Jika DFA berhenti pada *state* biasa atau *dead state*, program akan mengeluarkan *error* dan menjadikan token yang terbaca sebagai *unknown*. Lain halnya ketika DFA berhenti pada *final state*, program akan menjadikan token yang terbaca sesuai dengan jenis *state* terakhir yang dikunjungi.

DFA dimulai pada *start state* bernama q_{no_input}. *State* tersebut menandakan bahwa program belum menerima masukkan apapun kecuali *whitespaces*. Pada DFA ini, *whitespaces* adalah himpunan karakter yang memisahkan kandidat-kandidat token pada *source code*. Himpunan *whitespaces* berisi spasi, *newline*, *carriage return*, *horizontal tab*, dan karakter lain yang tergolong *whitespaces* (*whitespaces* = {‘ ‘,’\n’.’\r’,’\t’, …}), ketika program membaca *whitespaces*, program bertransisi *state* ke *state* yang sama seperti sebelumnya pada DFA.

Setelah *whitespaces* sudah terbaca semua, program akan membaca karakter pertama. Karakter pertama ini menentukan jenis/kandidat jenis token, seperti kebanyakan simbol yang merepresentasikan token berjenis simbol dan huruf akan mempresentasikan kandidat token berjenis *keyword* atau token *ident*. Karakter pertama yang diterima oleh DFA adalah *alphanumerical characters* (disingkat *an* di dalam DFA, dengan *an* = {A..Z,0..9}) dan simbol-simbol tertentu (*first_char* = *an* U {‘+’,’*’,’/’.’(‘.’)’,’[‘,’]’,’,’,’;’,’.’,’-’,’>’,’<’,’:’, ’=’,’{‘,’”’} [17 simbol]). Selain *first_char*, DFA akan berpindah *state* ke $q_{UNKNOWN}$. Perlu diperhatikan juga bahwa DFA beserta program implementasi bersifat *case-insensitive*, sehingga karakter ‘a’ dengan ‘A’ dianggap sama.

Simbol pada *first_char* pada umumnya dengan sendirinya bisa menjadi token. Beberapa gabungan dari dua simbol juga dapat diterima oleh DFA. Ada beberapa simbol seperti petik, *left curly brace*, dan simbol sama dengan yang memerlukan kelanjutan karakter sesuai pada DFA agar token diterima. Ketika pembacaan simbol diikuti oleh karakter apapun dan berada pada *final state*, DFA akan bertransisi ke $q_{UNKNOWN}$. Hal ini terkecuali pada simbol petik urutan genap yang bisa diikuti oleh petik lagi untuk membuat *string* yang lebih panjang selama petik tersebut ditutup kembali.

Ketika DFA menerima karakter pertama berupa alfabet dengan karakter pertama tersebut adalah karakter pertama dari suatu *keyword*, maka DFA akan berpindah *state* berjenis kandidat *keyword* yang ditandai dengan nama *state* berupa alfabet dari *substring* awalan *keyword* (seperti q_C, q_{PROGRA}, q_{EL}, dan sebagainya). *State-state* tersebut menandakan bahwa DFA telah membaca *substring* awal dari sebuah *keyword*. Seluruh *state* yang menunjukkan DFA telah membaca *substring* awalan dari suatu *keyword* merupakan *final state*

yang menghasilkan token *ident*. Token *keyword* didapat ketika DFA berhenti pada *state* dengan nama yang sama dengan *keyword* terkait (seperti q_{CASE}, $q_{PROGRAM}$, q_{ELSE}, dan sebagainya).

DFA juga akan berpindah ke q_{IDENT}, *state* yang menandakan token tersebut kandidat dari token *ident*, ketika di awal maupun tengah pembacaan karakter alfabet ternyata karakter terbaca tidak membentuk *keyword* manapun. Setelah masuk token *ident*, seluruh karakter *an* diterima oleh DFA sampai habis. Jika ditemukan karakter bukan *an*, maka DFA masuk ke $q_{UNKNOWN}$.

Selain simbol dan alfabet, angka juga diterima sebagai karakter pertama token. DFA menghasilkan token *intcon* ketika hanya membaca angka saja tanpa ada karakter lainnya (direpresentasikan oleh q_{INTCON}). Token *intcon* juga bisa didapat dengan pembacaan karakter pertama berupa '-' lalu diikuti oleh angka yang menandakan nilai token adalah negatif. Token *realcon* mirip seperti token *intcon*, hanya saja terdapat simbol titik di antara angka-angka yang dibaca (direpresentasikan oleh $q_{REALCON}$, aturan negatif berlaku juga sama seperti *intcon*).

DFA yang dibuat di atas hanya membaca karakter di *source code* sampai *whitespace* yang berada di setelah karakter bukan *whitespace* (kecuali saat membaca *string*, *char*, atau komentar) atau sampai karakter yang membuat DFA pindah ke $q_{UNKNOWN}$ namun merupakan kandidat token selanjutnya (misal, DFA akan membaca karakter sampai sebelum N input ==NOT). Ketika DFA selesai membaca karakter, DFA di-*reset* sehingga kembali ke *start state* dan memulai untuk membaca token selanjutnya.

2.2. Implementasi Program

2.2.1. Program Main

Program dimulai dengan menjalankan 2 argumen dalam terminal:
{program} {nama_file.txt}

Program main akan menerima sebuah file dalam bentuk.txt yang nantinya akan diubah ke dalam input file stream yang akan diproses ke dalam lexer.

2.2.2. Class Lexer

2.2.2.1. runLexer()

Program lexer merupakan implementasi desain DFA di mana lexer akan mengubah input file stream menjadi sebuah vector of object Token nantinya. Lexer bekerja dengan membaca char satu per satu mengikuti alur state dan transisinya.

```
vector<Token> Lexer::runLexer() {
    vector <Token> tokens;

    while (input.peek() != EOF) {

        // Ignore whitespace
        while (input.peek() != EOF &&
isspace(input.peek())) {
            input.get();
        }

        // Check for EOF after consuming whitespace
        if (input.peek() == EOF) {
            break;
        }

        // Grab char and check the current state
its in
        char ch = static_cast<char> (input.peek());

        if (isalpha(ch)){
            Token result = scanIndentOrKeyword();
            tokens.push_back(result);

        } else if (isdigit(ch)){
            Token result = scanNumber();
            tokens.push_back(result);

        } else if (ch == '\''){
            Token result = scanString();
            tokens.push_back(result);

        }  else if (ch == '(') {
            tokens.push_back(scanCommentParen());
        }   else if (ch == ':' || ch == ';' || ch
== '+' || ch == '<' || ch == '>' || ch == '*' || ch
== '/' ||
                    ch == '=' || ch == '-' || ch ==
'.' || ch == ',' || ch == ')' || ch == '[' || ch ==
']') {
            Token result = scanSymbol();
```

```
            tokens.push_back(result);
        } else if (ch == '{'){
            tokens.push_back(scanCommentCurly());
        }
        else {
            // any symbol that are not recongnized
as valid token will be pass as "unknown" token
            string unknownSymbol = string(1, ch);
            Token result = Token("unknown",
unknownSymbol);
            tokens.push_back(result);
        }

    }


    return tokens;
}
```

runLexer() merupakan sebuah method pertama yang akan di pass sehabis dari program main jalankan. runLexer() bekerja dengan membaca satu karakter dan menentukan alur state dan transisinya ke mana bergantung dengan karakter saat ini. Contoh, pada kasus jika sebuah karakter merupakan sebuah *whitespace* " ", lexer akan mengabaikan karakter tersebut sampai karakter yang dibaca sekarang sudah bukan *whitespace* lagi. Di sisi lain, jikalau sebuah karakter mengandung sebuah simbol yang valid (dalam artian karakter tersebut mengandung sebuah sekuens simbol valid dalam tabel token SYMBOLS) maka lexer akan memanggil method scanSymbol().

2.2.2.2. scanSymbol()

Merupakan method untuk mengecek transisi state untuk kasus karakter yang mengandung symbol :, ;, +, <, >, *, /, =. -, ., ,, (,), [,]. Dimana symbol akan ditelurusi lebih lanjut transisi alur statenya dan akan meng-*construct* Token nya dengan tipe yang valid.

2.2.2.3. scanNumber()

Jikalau karakter yang dibaca pertama mengandung sebuah digit 1, 2, 3, 4, 5, 6, 7, 8, 9, dan 0, maka scanNumber() akan dijalan dengan melakukan pembacaan karakter selama karakter tersebut masih merupakan sebuah digit integer (*intcon*). Jikalau pada proses transisi pembacaan terdapat sebuah tanda titik ".",

maka ada transisi akan berpindah dengan membaca digit untuk membuat sebuah token float (*realcon*). Jika masukkan karakter sudah tidak bukan digit lagi (terkecuali alphabet, di mana alphanumerical akan di-*pass* sebagai ident), maka alur transisi pada scanNumber akan stop dan akan membuat tokennya berdasarkan tipe token dari karakter yang sudah dibaca tersebut.

2.2.2.4. scanIndentOrKeyword()

Untuk karakter yang dibaca alphanumeric (karakter yang mengandung digit atau alphabet) akan diproses lebih lanjut dalam bagian token indent. Namun, jika sebuah kata setelah dibaca secara sekuens mengandung sebuah keyword spesial seperti program, var, begin, end, for, if, else, array, dsb. Maka karakter tersebut akan dimasukkan ke dalam token keyword sesuai dengan tipe keywordnya masing-masing.

2.2.3. Class Token

Class token berfungsi sebagai menyimpan hasil tokenisasi lexer yang akan disimpan dalam object berisi token-token yang akan disimpan nantinya. Ketika lexer setelah diselesaikan dan menghasilkan vector<Token>, program dapat melakukan pemanggilan method toString() untuk mengubah objek token yang sudah dihasilkan menjadi output print terformat sebagai hasil akhir

```
#include "Token.h"

Token::Token(const string& type, const string&
lexeme) : type(type), lexeme(lexeme) {

}

string Token::getType() const {
    return type;
}

string Token::getLexeme() const {
    return lexeme;
}

string Token::toString() const {
    string result = "";
    if (this->type == "intcon" || this->type ==
```

```
"realcon" || this->type == "charcon" || this->type
== "string" || this->type == "ident" || this->type
== "comment") {
        result += this->type;
        result += " (";
        result += this->lexeme;
        result += ")";
        return result;
    }
    else {
        return this->type;
    }
}
```

3. Pengujian

3.1. Test Case 1

`program Hello;`
 	
`var`
`  a, b: integer;`
 	
`begin`
`  a := 5;`
`  b := a + 10;`
`  writeln('R', b); {i huyh}`
`(*this guy*)`
`end.`
`'''`
	`programsy`
	`ident (Hello)`
	`semicolon`
	
	`varsy`
	`ident (a)`
	`comma`
	`ident (b)`
	`colon`
	`ident (integer)`
	`semicolon`
	
	`beginsy`
	`ident (a)`
	`becomes`
	`intcon (5)`
	`semicolon`
	
	`ident (b)`
	`becomes`
	`ident (a)`
	`plus`
	`intcon (10)`
	`semicolon`
	
	`ident (writeln)`
	`lparent`
	`charcon (R)`
	`comma`
	`ident (b)`
	`rparent`
	`semicolon`
	
	`comment (i huyh)`
	`comment (this guy)`
	`endsy`
	`period`
	`unknown ('')`

Bukti *screenshot*:

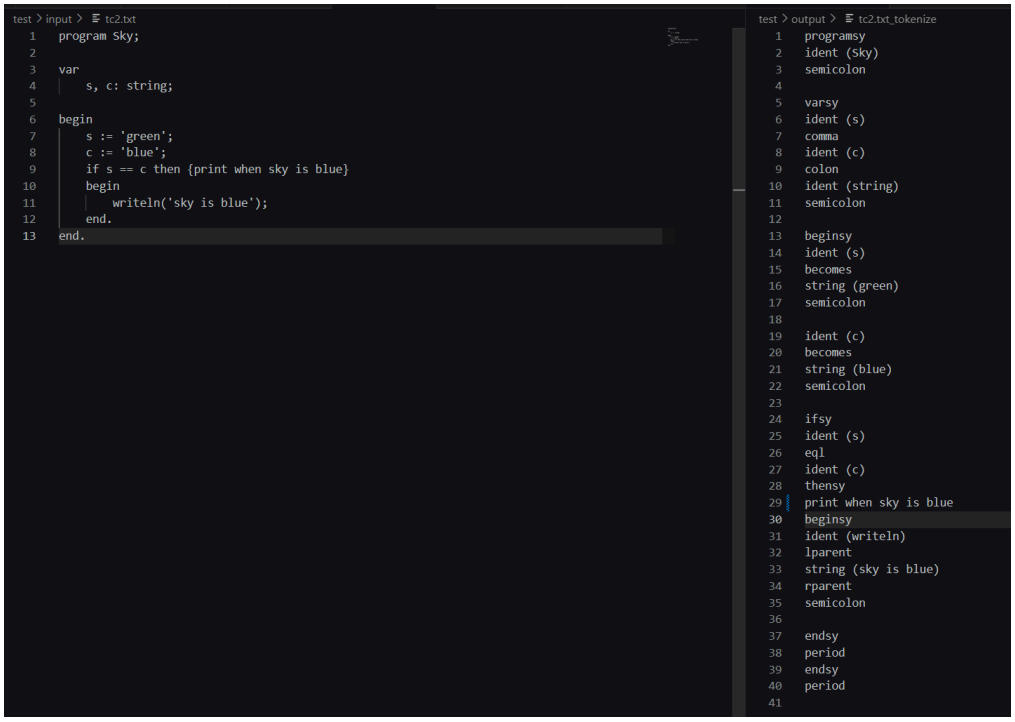
Gambar 2. Input dan Output Test Case 1

3.2. Test Case 2

`program Sky;`
 	
`var`
`  s, c: string;`
 	
`begin`
`  s := 'green';`
`  c := 'blue';`
`  if s == c then {print when sky is blue}`
`  begin`
`  writeln('sky is blue');`
`  end.`
`end.`
	`programsy`
	`ident (Sky)`
	`semicolon`
	
	`varsy`
	`ident (s)`
	`comma`
	`ident (c)`
	`colon`
	`ident (string)`
	`semicolon`
	
	`beginsy`
	`ident (s)`
	`becomes`
	`string (green)`
	`semicolon`
	
	`ident (c)`
	`becomes`
	`string (blue)`
	`semicolon`
	
	`ifsy`
	`ident (s)`
	`eql`
	`ident (c)`
	`thensy`
	`{print when sky is blue}`
	`beginsy`
	`ident (writeln)`
	`lparent`
	`string (sky is blue)`
	`rparent`
	`semicolon`

	endsy period endsy period

Bukti *screenshot*:



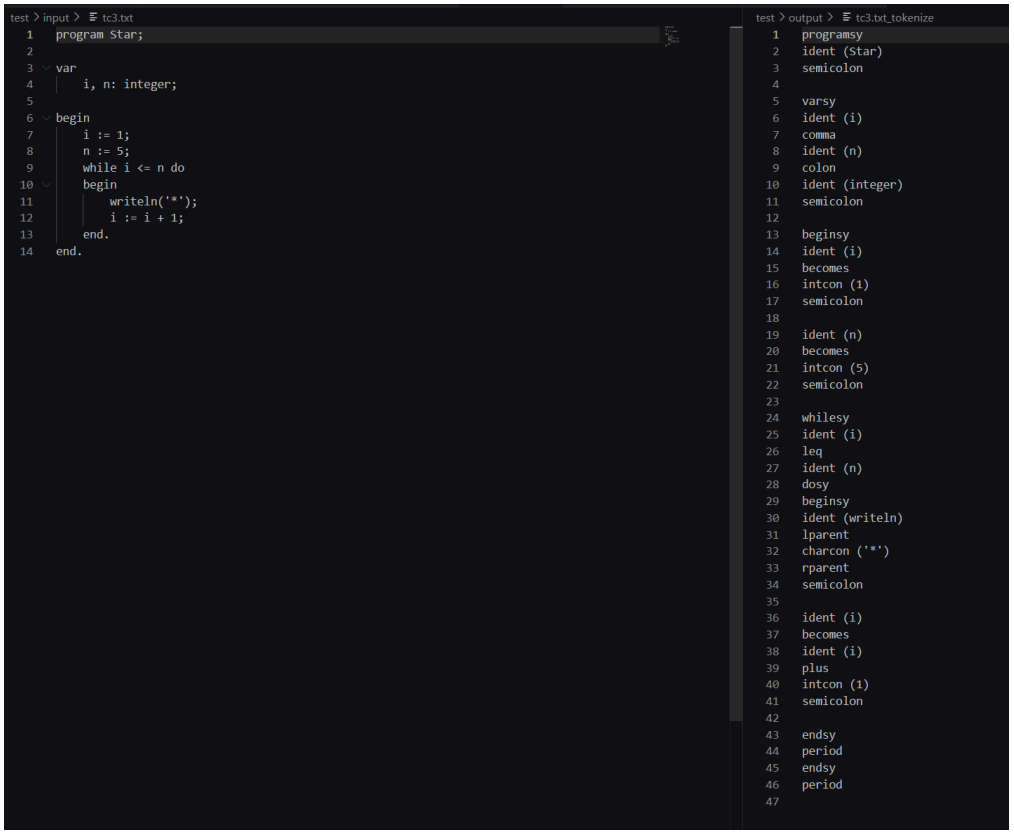
Gambar 3. Input dan Output Test Case 2

3.3. Test Case 3

program Star;
 	
var
  i, n: integer;
 	
begin
  i := 1;
  n := 5;
  while i <= n do
  begin
  writeln('*');
  i := i + 1;
  end.
end.
	programsy
	ident (Star)
	semicolon
	
	varsy
	ident (i)
	comma
	ident (n)
	colon
	ident (integer)
	semicolon
	
	beginsy
	ident (i)
	becomes
	intcon (1)
	semicolon
	
	ident (n)
	becomes
	intcon (5)
	semicolon

	whilesy ident (i) leq ident (n) dosy beginsy ident (writeln) lparent charcon ('*') rparent semicolon ident (i) becomes ident (i) plus intcon (1) semicolon endsy period endsy period

Bukti *screenshot*:



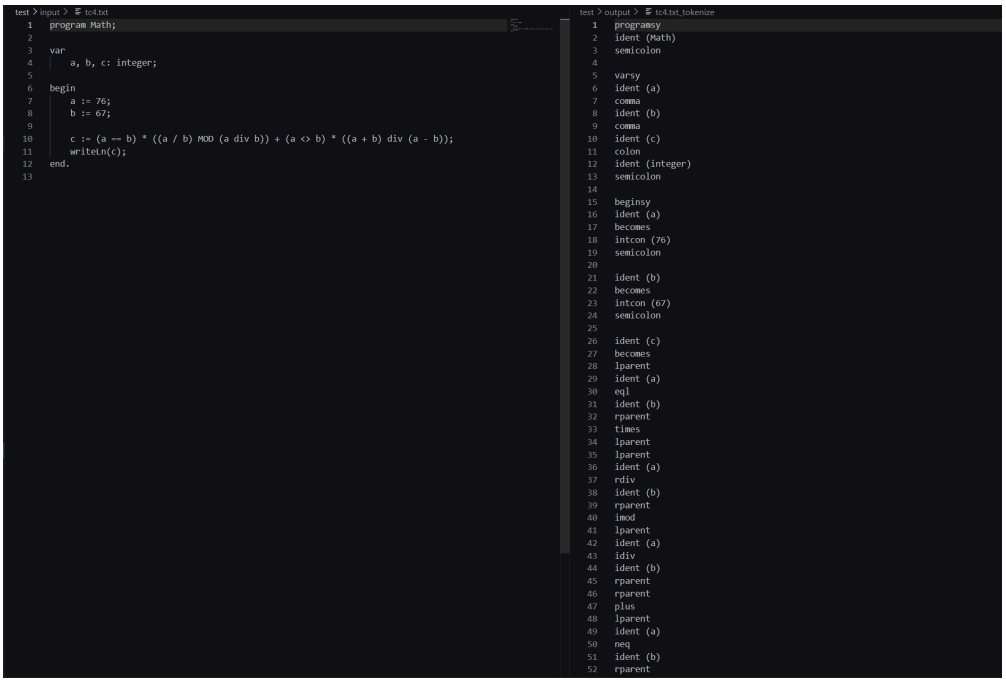
Gambar 4. Input dan Output Test Case 3

3.4. Test Case 4

`program Math;`
 	
`var`
`  a, b, c: integer;`
 	
`begin`
`  a := 76;`
`  b := 67;`
 	
`  c := (a == b) * ((a / b) MOD (a div b)) + (a <> b) * ((a + b) div (a - b));`
`  writeLn(c);`
`end.`
	`programsy`
	`ident (Math)`
	`semicolon`
	
	`varsy`
	`ident (a)`
	`comma`
	`ident (b)`
	`comma`
	`ident (c)`
	`colon`
	`ident (integer)`
	`semicolon`
	
	`beginsy`
	`ident (a)`
	`becomes`
	`intcon (76)`
	`semicolon`
	
	`ident (b)`
	`becomes`
	`intcon (67)`
	`semicolon`
	
	`ident (c)`
	`becomes`
	`lparent`
	`ident (a)`
	`eql`
	`ident (b)`
	`rparent`
	`times`
	`lparent`
	`lparent`
	`ident (a)`
	`rdiv`
	`ident (b)`
	`rparent`
	`imod`
	`lparent`
	`ident (a)`
	`idiv`
	`ident (b)`
	`rparent`
	`rparent`
	`plus`
	`lparent`
	`ident (a)`
	`neq`
	`ident (b)`
	`rparent`
	`times`
	`lparent`
	`lparent`
	`ident (a)`
	`plus`

	```ident (b) rparent idiv lparent ident (a) minus ident (b) rparent rparent semicolon ident (writeLn) lparent ident (c) rparent semicolon endsy period```

Bukti *screenshot*:



Gambar 5. Input dan Output Test Case 4

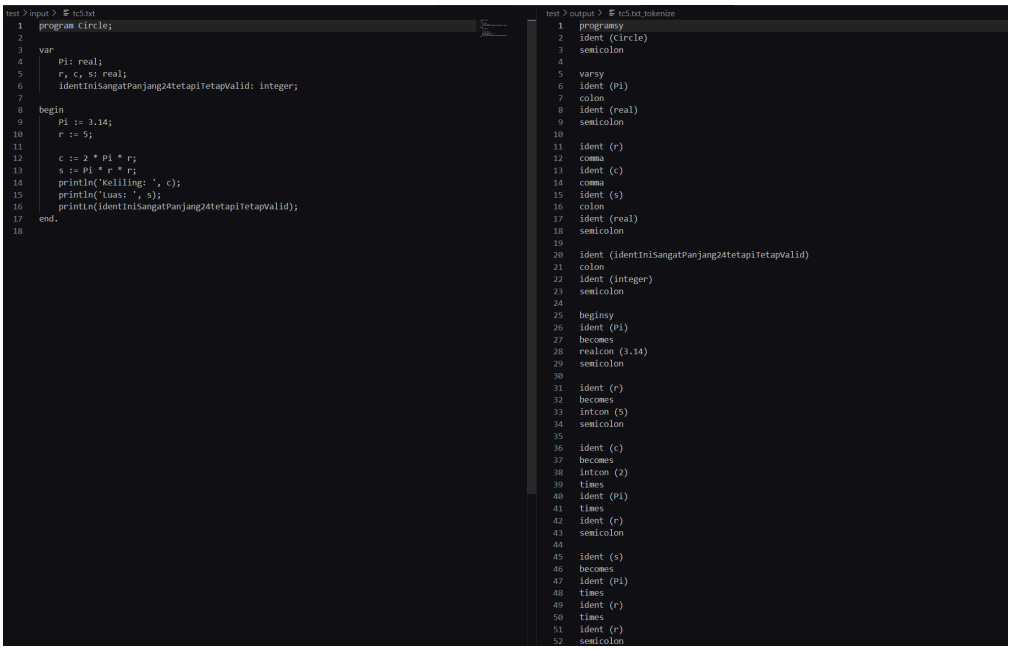
3.5. Test Case 5

``program Circle;
 	
var
  Pi: real;
  r, c, s: real;
 	
identIniSangatPanjang24tetapiTetapValid: integer;```
	``programsy
	ident (Circle)
	semicolon
	
	varsy
	ident (Pi)
	colon
	ident (real)
	semicolon```

begin
  Pi := 3.14;
  r := 5;
 	
  c := 2 * Pi * r;
  s := Pi * r * r;
  println('Keliling: ', c);
  println('Luas: ', s);
 	
printLn(identIniSangatPanjang24tetapiTetapValid);
end.
	
	ident (r)
	comma
	ident (c)
	comma
	ident (s)
	colon
	ident (real)
	semicolon
	
	ident (identIniSangatPanjang24tetapiTetapValid)
	colon
	ident (integer)
	semicolon
	
	beginsy
	ident (Pi)
	becomes
	realcon (3.14)
	semicolon
	
	ident (r)
	becomes
	intcon (5)
	semicolon
	
	ident (c)
	becomes
	intcon (2)
	times
	ident (Pi)
	times
	ident (r)
	semicolon
	
	ident (s)
	becomes
	ident (Pi)
	times
	ident (r)
	times
	ident (r)
	semicolon
	
	ident (println)
	lparent
	string (Keliling:)
	comma
	ident (c)
	rparent
	semicolon
	
	ident (println)
	lparent
	string (Luas:)
	comma

	ident (s) rparent semicolon ident (printLn) lparent ident (identIniSangatPanjang24tetapiTetapValid) rparent semicolon endsy period

Bukti *screenshot*:



Gambar 6. Input dan Output Test Case 5

4. Kesimpulan dan Saran

4.1. Kesimpulan

Dalam Milestone 1 Tugas Besar IF2224 Teori Bahasa Formal dan Otomata ini, kami berhasil mengimplementasikan *lexical analyzer* (*lexer*) untuk bahasa pemrograman Arion menggunakan *Deterministic Finite Automata* (DFA). Proses ini melibatkan pembangunan aturan DFA untuk mengenali berbagai jenis token, dengan token yang sudah dihasilkan, dibangun menjadi dengan format yang diharapkan. Pengimplementasian program *lexical analyzer* ini adalah salah satu contoh pengaplikasian *Deterministic Finite Automata* di kehidupan nyata, khususnya pada dunia komputasi.

4.2. Saran

Sebelumnya, kami sangat berterima kasih terhadap para asisten yang telah mempersiapkan tugas besar ini dengan penuh usaha. Kami juga sangat mengapresiasi para asisten karena telah menjawab pertanyaan-pertanyaan baik dari anggota kelompok kami maupun dari rekan-rekan kami di luar kelompok dengan respon yang cukup cepat. Kami menyarankan, untuk kedepannya, jika membuat tugas mengenai *lexical analyzer* untuk mengurangi *keyword* berbasis alfabet dan lebih utamakan *keyword* berbasis simbol. *Keyword* yang berbasis alfabet tersebut menjadikan desain DFA yang dibuat menjadi sangat besar sehingga menjadi sulit untuk melacak kebenaran pembuatan DFA..

5. Lampiran

5.1. Github
https://github.com/Naufal-Pinasthika/TBF-Tubes-IF2224-2026

5.2. Diagram DFA
DFA 13.png

5.3. Tabel Pembagian Tugas

Nama	NIM	Workload	Persentase
Faiq Azzam Nafidz	13524003	- Desain DFA - Penjelasan DFA - Bug fixing pada Lexer.cpp dan Token.cpp	23%
Anindya Naufal Pinasthika	13524013	- Token.cpp - Lexer.cpp(partial) - main.cpp(partial) - Teori Singkat	35%
Mahmudia Kimdaro Amin	13524083	- Token.cpp - Lexer.cpp(partial) - main.cpp(partial)	21%
Jingglang Galih Rinenggan	13524095	- Lexer.cpp(partial) - Pengujian - README	21%

6. Referensi

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson. Tersedia di: https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf (Diakses pada 2 April 2026)

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson/Addison-Wesley.